

Program 16:

```
vedsk0831@DESKTOP-83FU7HB:~$ mkdir -p ~/OSSP/Program16
cd ~/OSSP/Program16
vedsk0831@DESKTOP-83FU7HB:~/OSSP/Program16$ pwd
/home/vedsk0831/OSSP/Program16
vedsk0831@DESKTOP-83FU7HB:~/OSSP/Program16$ nano program16.c
vedsk0831@DESKTOP-83FU7HB:~/OSSP/Program16$ gcc program16.c -o program1
vedsk0831@DESKTOP-83FU7HB:~/OSSP/Program16$ ls
program16  program16.c
vedsk0831@DESKTOP-83FU7HB:~/OSSP/Program16$ ./program16
Simple Background Job Shell
Type 'exit' to quit
Use '&' at the end of a command for background execution
shell> sleep 5
shell> sleep 10 &
[1] 642 running in background
shell> jobs

----- Job Table -----
[1] PID=642  Running   sleep 10
-----
shell> jobs
[1] Done           sleep 10

----- Job Table -----
[1] PID=642  Completed sleep 10
-----
shell> sleep 5 &
sleep 10 &
sleep 15 &[2] 645 running in background
shell> [3] 646 running in background
shell>
[4] 647 running in background
shell> |
```

Code

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
#include <errno.h>
```

```
#define MAX_JOBS 100
```

```
#define MAX_CMD 256
```

```
typedef struct {
```

```
    int job_id;
```

```
    pid_t pid;
```

```
    char command[MAX_CMD];
```

```
    int running;
```

```
} Job;
```

```
Job jobs[MAX_JOBS];
```

```
int job_count = 0;
```

```
int next_job_id = 1;
```

```
/* Add a new background job */
```

```
void add_job(pid_t pid, const char *command)
```

```
{
```

```
    if (job_count >= MAX_JOBS) {
```

```
        printf("Job table full!\n");
```

```
        return;
```

```
    }
```

```
    jobs[job_count].job_id = next_job_id++;
```

```
    jobs[job_count].pid = pid;
```

```
    strncpy(jobs[job_count].command, command, MAX_CMD - 1);
```

```
    jobs[job_count].command[MAX_CMD - 1] = '\0';
```

```
    jobs[job_count].running = 1;
```

```
    printf("[%d] %d running in background\n",
```

```

        jobs[job_count].job_id,
        jobs[job_count].pid);

    job_count++;
}

/* Check whether background processes have completed */
void update_jobs()
{
    for (int i = 0; i < job_count; i++) {

        if (!jobs[i].running)
            continue;

        int status;

        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG
        );

        if (result == jobs[i].pid) {

            jobs[i].running = 0;

            if (WIFEXITED(status)) {
                printf("[%d] Done\t%s\n",

```

```

        jobs[i].job_id,
        jobs[i].command);
    }
    else if (WIFSIGNALED(status)) {
        printf("[%d] Terminated\t%s\n",
            jobs[i].job_id,
            jobs[i].command);
    }
}
}
}
}

```

/* Display job table */

```
void show_jobs()
```

```
{
```

```
    update_jobs();
```

```
    printf("\n----- Job Table ----- \n");
```

```
    if (job_count == 0) {
```

```
        printf("No jobs.\n");
```

```
        return;
```

```
    }
```

```
    for (int i = 0; i < job_count; i++) {
```

```
        printf("[%d] PID=%d %s %s\n",
```

```
            jobs[i].job_id,
```

```

        jobs[i].pid,
        jobs[i].running ? "Running" : "Completed",
        jobs[i].command);
    }

    printf("-----\n");
}

int main()
{
    char input[MAX_CMD];

    printf("Simple Background Job Shell\n");
    printf("Type 'exit' to quit\n");
    printf("Use '&' at the end of a command for background execution\n");

    while (1) {

        update_jobs();

        printf("shell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';
    }
}

```

```
if (strlen(input) == 0)
    continue;

/* Exit */
if (strcmp(input, "exit") == 0) {
    break;
}

/* Display jobs */
if (strcmp(input, "jobs") == 0) {
    show_jobs();
    continue;
}

/*
 * Detect '&'
 */
int background = 0;

int len = strlen(input);

if (len > 0 && input[len - 1] == '&') {

    background = 1;

    input[len - 1] = '\0';

    /* Remove trailing spaces */
```

```

while (len > 1 &&
    input[len - 2] == ' ') {
    input[len - 2] = '\0';
    len--;
}
}

pid_t pid = fork();

if (pid < 0) {
    perror("fork");
    continue;
}

/* Child process */
if (pid == 0) {

    char *args[64];

    int argc = 0;

    char *token = strtok(input, " ");

    while (token != NULL && argc < 63) {

        args[argc++] = token;
        token = strtok(NULL, " ");
    }

```

```
args[argc] = NULL;

if (argc == 0)
    exit(0);

execvp(args[0], args);

perror("execvp");
exit(1);
}

/* Parent process */
else {

    if (background) {

        add_job(pid, input);

        /*
         * Do NOT wait.
         * Shell immediately returns to prompt.
         */

    } else {

        /*
         * Foreground process.
         * Parent waits for child.
```

```

        */

        waitpid(pid, NULL, 0);
    }
}

/*
 * Wait for remaining background jobs
 */
printf("\nWaiting for remaining jobs...\n");

for (int i = 0; i < job_count; i++) {

    if (jobs[i].running) {
        waitpid(jobs[i].pid, NULL, 0);
        printf("[%d] Completed\t%s\n",
            jobs[i].job_id,
            jobs[i].command);
    }
}

printf("Shell terminated.\n");

return 0;
}

```